Detailed Design Document (LLD)

Edge Industrial AI Defect Inspection, Optical Self-Healing & Vector-RAG Telemetry System

Framework: MASTER_SDLC_AGENTS | Version 1.0.0 | Phase 7: DevOps Release | Date: October 2026

1. Executive Summary & Autonomous Governance

The Edge Industrial AI Defect Inspection Platform is a production-grade automated visual inspection and work order synthesis system engineered for discrete metal manufacturing lines (milled aluminum, cast iron, stamped galvanized steel, titanium alloys). Operating with sub-100ms latency on standard edge x86/ARM CPUs without requiring GPU hardware, the platform combines grain-neutral row intensity subtraction, a 21-dimensional feature descriptor engine, 100-tree Random Forest classification, and Vector Retrieval-Augmented Generation (RAG) to synthesize OSHA 29 CFR 1910.147 Lockout/Tagout maintenance work orders upon flaw detection.

Autonomous Governance via MASTER_SDLC_AGENTS Framework:
This project was developed, verified, and released under the strict governance of the 7-Agent Master SDLC Framework with Human-in-the-Loop (HITL) quality gates across all 7 lifecycle phases:
• Phase 1 (Product Manager): Formulated PRD.md with plain-English terminology and bracketed operator tooltips.
• Phase 2 (Technical Architect): Authored architecture-spec.md establishing 2-tier ASCII presentation vs. inference engine.
• Phase 3 (Tech Lead): Authored task-queue.md defining modular tasks and Definition of Done.
• Phase 4 (Software Engineer): Implemented src/utils.py, src/detector.py, and src/app.py.
• Phase 5 (QA Test Engineer): Validated 34/34 passing unit and integration tests; triaged and closed 5 defects.
• Phase 6 (Security Auditor): Conducted SAST static analysis, hardened against decompression bombs, and certified SAFE TO COMMIT.
• Phase 7 (DevOps Engineer): Packaged production Docker container, manuals, and customer deliverables.

2. Architectural Pattern: Detective vs. Painter

The application architecture strictly decouples mathematical perception from graphical presentation through the Detective vs. Painter design pattern:
• The Detective (src/detector.py): Optical quality assessment (Laplacian variance, contrast, brightness), autonomous self-healing (unsharp mask, CLAHE, gamma LUT), 21-D feature extraction (geometry, Hu moments, GLCM texture), Random Forest classification, dynamic severity rating (0.0 to 10.0), and TF-IDF Vector-RAG work order synthesis.

• The Painter (src/utils.py): Grain-neutral row-wise mean intensity background cancellation, semi-transparent Crimson Red flaw overlays (RGB 239, 68, 68), bounding boxes, top status HUD banners, and RFC 4180 CSV telemetry export.
• The Operator Cockpit (src/app.py): Reactive Streamlit industrial dashboard with dual-viewport canvas, KPI cards, bracketed parameter sliders, and 1-click work order console.

3. System Contracts & Telemetry Data Dictionary

Field Name	Data Type	Units / Range	Industrial Description
defect_type	string	normal|scratch|crack|corrosion|dimensional	Classified defect state
severity_score	float	0.0 to 10.0	Continuous flaw severity rating (Normal = 0.0)
severity_level	string	Negligible|Minor|Moderate|Critical	Operational triage category
confidence	float	0.00 to 1.00	Random Forest ensemble class probability
optical_health	string	Optimal|Restored (Blur)|Restored (Glare)	Camera sensor operational integrity status
laplacian_var	float	Score (> 45 = Sharp)	High-frequency image gradient sharpness index
total_area	float	Pixels	Accumulated defect surface area in pixels
work_order_id	string	WO-M<frame>-<DEFECT>	Unique synthesized maintenance ticket identifier

4. Operator Plain-English Parameter Specifications

Dashboard Slider	Plain-English Guidance Tooltip	Factory Operational Impact
Defect Sensitivity Threshold	[Defect Sensitivity: Higher values catch microscopic hairlines but may increase false	Controls row-subtraction binarization offset (range: 10 to 60,

	alarms; lower values ignore minor cosmetic marks]	default 28).
Surface Contrast Boost	[Contrast Boost: Enhances surface texture contrast; recommended for dull, unpolished or dark-milled cast iron plates]	Adjusts CLAHE tile grid clipping limit for deep defect shadow enhancement.
Camera Sharpness Alarm	[Camera Sharpness Alarm: Flags blurry or dirty camera lens if variance falls below threshold; triggers auto-sharpening unsharp mask]	Governs Laplacian blur threshold triggering automatic self-healing high-boost filter.

5. Comparative Architectural Guide: Random Forest vs. ResNet vs. MobileNet vs. YOLO

A critical engineering decision in industrial vision design is selecting the optimal model architecture balancing edge inference latency, training sample requirements, texture-handling capability, and deployment cost:

Dimension	Random Forest + 21-D (Chosen)	ResNet-18/50	MobileNet-V3	YOLO-v8/v11
Inference Speed	< 15ms (Ultra Fast on CPU)	45ms - 85ms	20ms - 35ms	40ms - 90ms
Edge Hardware	Standard x86 / ARM CPU	Requires GPU / VPU	Edge NPU / Fast CPU	Requires CUDA GPU
Brushed Metal Grain	Immune (Row-neutral filter)	Moderate (Can trigger false alarms)	Prone to noise	Moderate
Data Requirements	Few-shot / Synthetic (< 100 samples)	Requires 5,000+ labeled images	Requires 3,000+ images	Requires 10,000+ bounding boxes
Memory Footprint	< 250 MB RAM	400 MB - 800 MB	200 MB - 350 MB	500 MB - 1.2 GB
Self-Healing RAG Link	Direct feature integration	Requires external embedding head	Requires embedding head	Object box bounding only

6. Defensive Security Architecture & Vulnerability Mitigations

1. Decompression Bomb Protection: Capped Image.MAX_IMAGE_PIXELS = 50_000_000 to neutralize pixel bomb DoS attacks.
2. Zero Arbitrary Code Execution: Zero instances of eval, exec, os.system, or subprocess. Pure in-memory execution.
3. CSV Formula Injection Defense: Escapes dangerous trigger prefixes (=, +, -, @) with a single-quote prefix (') per RFC 4180.
4. Model Deserialization Fallback: Encapsulates joblib.load() with automatic fallback to native on-the-fly training (<1s).
5. 24/7 Memory Isolation: Bounded memory footprint (< 250 MB RAM) ensures uninterrupted operation on factory lines.

7. Real-World Edge-Case Resolutions & Closed Defects (BUG-001 to BUG-005)

Defect ID	Subsystem	Root Cause	Resolution & Verification
BUG-001	src/app.py	Right header text wrapped into 2 lines, pushing mask down 28px	Created .viewport-header with white-space: nowrap; locked y=248px
BUG-002	src/utils.py	Unhandled shape unpacking on 2D grayscale arrays	Used img_arr.shape[:2] with conditional channel handling
BUG-003	src/detector.py	Mixed query casing lowered TF-IDF cosine similarity	Enforced .lower().strip() normalization across corpus and queries
BUG-004	src/detector.py	Normal flawless component computed non-zero severity	Enforced strict zero severity return when defect == 'normal'
BUG-005	src/detector.py	Severity score on massive defects exceeded 10.0 ceiling	Added mathematical np.clip(severity, 0.0, 10.0)